17/10/2026

Assessment Tool-2

(CO-1)

Name: Rashmitha Senthil Kumar

Reg No: 192525034

Course: Compiler Design (CSA1441)

Q1. a) Errors with descriptions

Error	Reason / correction
float x = 3.5	Missing ;
y = num1 +10	Missing ;
if (y==35 {	Missing)
printf("Correct value\n")	Missing ;
arr[3]	valid indices are 0-2. Use arr[2].
*p=20]	pointer is uninitialized.
10/0	Integer division by zero causes undefined behaviour.

b)

Error	Type
1. float x=3.5	Syntax
2. Y = num+10	Syntax
3. if (X==35 {	Syntax
4 Printf("Correct value(n")	Syntax
5. arr[3]	Runtime
6. *p=20	Runtime
7. 10/0	Arithmetic/runtime

c. Error Detection:

* Compilation:- Missing semicolons and) are detected during parsing.

* Runtime: Array access, pointer misuse, and division by zero cause undefined behaviour when executed.

* Compilers may also warn about these unsafe operations before execution.

d) Corrected program:

```
#include <stdio.h>
int main(void){
    int num1=25;
    float x=3.5f;
    int y;

    y=num1+10;
    if(y==35){
        Printf("Correct Value\n");

    }

int arr[5]={1,2,3};
printf("%d\n", arr[2]);
int value;
int *p=&value;
*p=20;
int divisor=2;
if(divisor!=0){
    int z=10/divisor;
    printf("%d\n",z);

}
while(num1<30){
    printf("%d\n",num1);
    num1++;
}
return 0;

}
```

e) Impact of division by zero and pointer misuse :

* Division by zero : causes undefined behaviour ; the program may terminate.

* Pointer misuse :- May corrupt memory or crash the program.

* Neither operation has a garanteed result.

f) Improvements in compiler design :

* Warn about uninitailized pointers.

* Detect constant out - of - bounds indices.

* Detect division by zero.

* Provide clear error locations and suggestion

* Support runtime safety checks.

Q2. Advanced lexical Error identification:-

a) The invalid forms :

5nums , value$, 10#20 , data@1 , 0xH12 , and 3.4.5.

b) Explain the violated rules.

Invalid form	Violated Rule
* 5 num	Identifier can't start with a digit.
* Value &	& can't be part of an identifier.
* 10#20	# cannot act as an arithmetic operator.
* data@1	@ is invalid in an ordinary C identifier.
* 0xH12	Hexadecimal digits must be 0-9, A-F or a-f.
* 3.4.5	This is not a valid floating constant.

c) Rewrite the invalid forms:

```
int num5 = 10;
float value = 8.5f;
int total = 10+20;
int data1 = 25;
printf("Sample Program");
int y = 0xA12;
float f = 3.45f;
char str[] = "Hello";
```

d) How the lexical analyzer handles errors:

* Reads characters and forms tokens.
* checks them against identifier, constant, and literal rules.
* Reports malformed tokens with their locations.
* Recovers and continues scanning.
* Passes valid token to the parser for syntax checking.

e) DFA rules for digit-starting identifiers:

State	Input	Next State
Start	Digit	Number
Number	Digit	Number
Number	letter or _	Invalid
Invalid	letter, digit, or _	Invalid
Invalid	Delimiter	Report error

f) Improvements in error reporting:

* Show line and column numbers.
* Display the invalid token.
* Explain the reason.

Q3. Intermediate Code and optimization Analysis :-

a) Identify errors and efficiencies :

1. a * b is calculated repeatedly.
2. b * a repeats the same integer product.
3. if (a = 5) assigns instead of comparing.
4. arr[2] is outside the two-element array.
5. ptr is uninitialized before dereferencing.
6. c is calculated but never used.

b) classify according to compiler phases :

Issue	classification
Repeated multiplication	Optimization opportunity
if (a = 5)	Logical error if comparison was intended.
arr [2]	Runtime Safety error
* ptr = 10	Runtime Safety error
unused c	Dead-code optimization opportunity.

c. Three - address code :

$t_1 = a * b$

$t_2 = a * b$

$t_3 = t_1 + t_2$

$t_4 = b * a$

$c = t_3 + t_4$

d. DAG representation and optimization :

Use one shared multiplication node :

Node	Value
N1	a
N2	b
N3	N1 * N2
N4	N3 + N3
N5, Labeled c	N4 + N3 .

e) Optimized target code:

```
MOV R0, #150
Store c, R0
```

f) Runtime Impact:

* Array violation:- Accesses memory outside the array.

* Pointer misuse:- writes through an invalid pointer.

* Both cause undefined behaviour, possibly memory corruption or a crash.

Corrections:-

```
printf("%d\n", arr[i]);
int value;
int *ptr = &value;
*ptr = 10;
```

Q4. Error Identification and classification:

a) Identify the error:

```
int x = 10
```

Here, the semicolon is missing at the end.

b) classify the error:

It is a syntax error because a declaration must end with ;. The complier detects its during parsing. No other error is evident.

c) Corrected program:

```
# include <stdio.h>
int main(void){
    int x=10;
    int y=20;
    int z=x+y;
    printf("%d \n", z);
    return 0;
}
```

Expected output:

30.

Q5. Compiler Error Analysis:

a) Identify the runtime errors:-

1. out-of-bounds access using arr[3].

2. Dereferencing the uninitialized pointer. ptr.

b) Array error :-

arr[2] declare two elements. only arr[0] and arr[1] are valid ; arr[3] is outside the array.

Pointer error :

ptr has not been assigned a valid address.

Therefore, *ptr = 50 attempts an invalid memory write.
Both cause undefined behaviour.

c) Corrected program :

```
#include <stdio.h>
int main(void){
    int arr[2] = {5,10};
    printf("%d\n", arr[1]);
    int value;
    int *ptr = &value;
```

```
    *ptr = 50;
    printf("%d\n", *ptr);

    return 0;

}
```

Expected output:

```
10
50
```